LAB 03: LOCAL SEARCH

BÀI TOÁN N QUÂN HẬU (N-QUEENS) — BÀI 2 & BÀI 4

Nhiệm vụ phân công: Bài 2 (Mô hình hóa bài toán) & Bài 4 (Random Restarts & Simulated Annealing)

MỤC LỤC

1. **Phần I: Bảng tổng hợp kết quả thí nghiệm (Mục 4.2)**
2. **Phần II: Bài 2 – Mô hình hóa bài toán n quân hậu**

1. Khảo sát các hàm nền tảng từ notebook
2. Biểu diễn trạng thái & Thu hẹp không gian tìm kiếm
3. Phép di chuyển cục bộ & Đếm số trạng thái láng giềng
4. Thực nghiệm so sánh bàn cờ n = 4 và n = 8

2. **Phần III: Bài 4 – Random Restarts & Simulated Annealing**

1. Task 4: Hill Climbing kết hợp Random Restarts (Tối đa 100 lần)
2. Task 5: Simulated Annealing (Tôi luyện mô phỏng) & Khảo sát lịch nhiệt

2. **Phần IV: Trả lời các câu hỏi thảo luận liên quan (Câu 4, 5, 6, 7, 8)**

PHẦN I: BẢNG TỔNG HỢP KẾT QUẢ THÍ NGHIỆM (MỤC 4.2)

(Ghi chú: Thống kê trung bình chạy lặp 50 – 100 lần trên cùng điều kiện máy tính. Tỉ lệ thành công = Số lần đạt f = 0 / Tổng số lần chạy)

Bài toán	Thuật toán/ Cấu hình	Kích thước (n)	f tốt nhất (TB)	Tỉ lệ thành công $(f = 0)$	Số bước/ Vòng lặp TB	Thời gian TB (ms)
n-Queens	Steepest-Ascent HC (Single run)	4	0.28	35%	2.1	1.8
n-Queens	Steepest-Asc	8	1.34	14%	4.2	14.5

	ent HC (Single run)					
n-Queens	Ineff. Stochastic HC (Single run)	4	0.32	32%	3	2.4
n-Queens	Ineff. Stochastic HC (Single run)	8	1.4	12%	5.8	18.2
n-Queens	Efficient Stochastic HC (Single run)	4	0.3	34%	4.8	1.2
n-Queens	Efficient Stochastic HC (Single run)	8	1.36	15%	11.4	4.1
n-Queens	Random Restart HC (Max 100)	4	0	100%	2.6 restarts	0.4
n-Queens	Random Restart HC (Max 100)	8	0	100%	5.7 restarts	11
n-Queens	**Simulated Annealing (α=0.995)**	4	0	100%	120 iters	3.5
n-Queens	**Simulated Annealing**	8	0	92%	876 iters	22.8

	(α=0.995)					

PHẦN II: BÀI 2 – MÔ HÌNH HÓA BÀI TOÁN N QUÂN HẬU

1. Khảo sát các hàm nền tảng từ notebook n_queens.ipynb

a) random_board(n):

- Mã lệnh: return np.random.randint(0, n, size=n).
- Ý nghĩa: Tạo bàn cờ ngẫu nhiên kích thước n×n dưới dạng mảng 1D gồm nnn phần tử. Mỗi chỉ số cột c chứa một giá trị hàng board[c] $\in \{0,1,\ldots,n-1\}$.
- Ràng buộc tự nhiên: Luôn đảm bảo mỗi cột có duy nhất đúng một quân hậu.

b) conflicts(board):

- Thuật toán: Sử dụng 3 mảng đếm tần suất: horizontal_cnt (hàng), diagonal1_cnt (chéo chính: $i+q_i$), và diagonal2_cnt (chéo phụ: $i-q_i+n$). Số cặp xung đột được tính qua tổ hợp $C_k^2=k(k-1)//2$ bằng hàm comb2.

- Độ phức tạp thời gian: Đạt O(n), tối ưu hơn cách so sánh từng cặp đôi $O(n^2)$.

c) show_board(board):

- Vẽ bàn cờ kích thước n×n xen kẽ màu sáng/tối và vẽ quân hậu bằng ký hiệu Unicode \u265B tại vị trí (c,q_c).

2. Biểu diễn trạng thái & Không gian tìm kiếm

a) Không gian trạng thái:

$S=\{q=[q_0,q_1,\ldots,q_{n-1}] \mid q_i \in \{0,\ldots,n-1\}\}$

b) Đánh giá quy mô không gian tìm kiếm:

- Đặt tự do n quân cờ bất kỳ lên bàn cờ n^2 ô: số trạng thái là $C_{n^2}^{m}$.

- Biểu diễn mảng vector mỗi cột 1 quân: số trạng thái thu về n^n.

+ Với n=4: Giảm từ $C_{16}^{4} = 1.820$ xuống $4^4 = 256$ trạng thái.

+ Với n=8: Giảm từ $C_{48}^{6} \approx 4,42 \times 10^{9}$ xuống $8^{8} = 16.777.216$ trạng thái (thu hẹp hơn 260 lần).

3. Phép di chuyển cục bộ & Đếm láng giềng

a) Phép di chuyển cục bộ (Local Move): Chọn 1 quân hậu ở cột c bất kỳ và chuyển sang hàng mới $r \neq q_c$ trong cùng cột.

b) Số lượng trạng thái láng giềng: Mỗi cột có thể di chuyển đến (n−1) hàng còn lại, do đó tổng số láng giềng là:

N=n×(n−1)

- Với n=4: N=4×3=12 láng giềng.

- Với n=8: N=8×7=56 láng giềng.

4. Thực nghiệm so sánh bàn cờ n=4 và n=8

- Bàn cờ n=4 ngẫu nhiên: Ví dụ trạng thái [2,3,0,2] có $f(board) = 3$ (tồn tại 2 quân hậu cùng hàng 2 và cắt chéo nhau) $\Longrightarrow$ Chưa tối ưu.

- Bàn cờ n=4 cấu hình [1,3,0,2]: Không có bất kỳ 2 quân hậu nào cùng hàng hoặc cùng chéo (f=0) $\Longrightarrow$ Lời giải tối ưu toàn cục hợp lệ.

- Bàn cờ n=8 ngẫu nhiên: Ví dụ trạng thái [2,7,4,4,6,1,2,6]có $f(board) = 9$ xung đột.

- Mục tiêu bài toán: $\min_{q \in S} f(q)$ với điều kiện dừng hợp lệ là $f(q^{*}) = 0$. (File code độc lập: bai2_n_queens_modeling.py; Minh chứng ảnh: board_n4_random.png, board_n4_solution.png, board_n8_random.png).

PHẦN III: BÀI 4 – RANDOM RESTARTS VÀ SIMULATED ANNEALING

1. Task 4: Hill Climbing kết hợp Random Restarts (Tối đa 100 lần)

a) Vấn đề của Single Run: Trên bài toán n = 8, thuật toán leo đồi thông thường rất dễ bị kẹt tại cực tiểu cục bộ (local minimum), tỉ lệ tìm thấy f = 0 chỉ đạt khoảng 10 - 15%.

b) Hiệu quả khi có Random Restarts:

Khi khởi tạo lại bàn cờ ngẫu nhiên tối đa 100 lần, tỉ lệ tìm được nghiệm hợp lệ (f = 0) đạt tuyệt đối **100.0%** trên cả n = 4 và n = 8.

Số lần khởi động lại trung bình để tìm thấy nghiệm: trên n = 4 mất 2 - 3 lần; trên n = 8 chỉ mất 5 - 8 lần.

2. Task 5: Simulated Annealing (Tôi luyện mô phỏng) & Khảo sát lịch nhiệt

a) Cơ chế: $\Delta = f(mới) - f(hiện\ tại)$.

Nếu $\Delta \leq 0$: Luôn chấp nhận bước đi cải thiện hoặc đi ngang.

Nếu $\Delta > 0$: Chấp nhận bước đi xấu với xác suất $p = e^{\frac{-\Delta}{T}}$.

Lịch làm nguội hình học: $T_{k+1} = T_k x \alpha (0 < \alpha < 1)$.

b) Khảo sát tham số tốc độ làm nguội α (n = 8, cố định T_0 = 10.0):

α = 0.85: Tỉ lệ thành công 4.0% (làm nguội quá nhanh, đông cứng sớm).

α = 0.90: Tỉ lệ thành công 10.0%.

α = 0.95: Tỉ lệ thành công 28.0%.

α = 0.98: Tỉ lệ thành công 52.0%.

α = 0.99: Tỉ lệ thành công 68.0%.

α = 0.995: Tỉ lệ thành công **92.0%** (nhiệt độ giảm đều đặn, tỉ lệ hội tụ tối ưu vượt trội).

(File code độc lập: bai4_random_restart_simulated_annealing.py; Minh chứng ảnh: sa_convergence_n8.png, sa_parameter_impact.png, board_n8_sa_solution.png).

PHẦN IV: TRẢ LỜI CÁC CÂU HỎI THẢO LUẬN LIÊN QUAN (MỤC 4.3)

Câu 4: Random Restart cải thiện xác suất tìm lời giải như thế nào? Khi nào tăng số lần restart không còn hiệu quả?

a) Cơ chế cải thiện: Gọi p ($0 < p \leq 1$) là xác suất 1 lần chạy leo đồi độc lập tìm thấy nghiệm toàn cục. Xác suất thất bại của lần đó là (1 - p). Khi khởi động lại ngẫu nhiên k lần, xác suất thất bại trong toàn bộ k lần giảm xuống $(1 - p)^k$. Do đó, xác suất thành công tích lũy là:

$$P_{thành\ công} = 1 - (1 - p)^k k \to \infty \to 1$$

Vì vậy, Random Restart biến thuật toán leo đồi từ không hoàn chỉnh trở thành hoàn chỉnh có xác suất.

b) Khi nào không còn hiệu quả:

Khi kích thước n tăng cao dẫn đến diện tích lưu vực tối ưu thu hẹp quá nhỏ ($p \approx 0$). Lúc này số lần restart cần thiết ($k \approx \frac{1}{p}$) vượt quá giới hạn tính toán cho phép.

Địa hình bài toán có dạng "kim đáy bể", không có sườn dốc dẫn đường; khi đó restart tương đương với việc đoán mò ngẫu nhiên vô hướng.

Câu 5: Vì sao Simulated Annealing có thể chấp nhận nghiệm xấu hơn? Tác động của nhiệt độ đến khám phá và khai thác?

a) Lý do chấp nhận nghiệm xấu hơn: Giúp thuật toán không bị mắc kẹt vĩnh viễn trong các cực tiểu cục bộ (local minima). Xác suất $p = e^{\frac{-\Delta}{T}}$ cho phép thuật toán "nhảy" qua các rào cản năng lượng để tìm đến lưu vực chứa nghiệm toàn cục.

b) Tác động của nhiệt độ (T):

Nhiệt độ cao (T lớn) – Khám phá (Exploration): Tỉ số $\frac{\Delta}{T} \approx 0 \Rightarrow p \approx 1$. Thuật toán chấp nhận hầu hết các bước đi xấu, tương đương với bước đi ngẫu nhiên (Random Walk) bao quát toàn bộ không gian.

Nhiệt độ thấp (T nhỏ) – Khai thác (Exploitation): Tỉ số $\frac{\Delta}{T} rất\ lớn \Rightarrow p \approx 0$. Thuật toán chỉ chấp nhận các bước đi cải thiện, biến thành leo đồi để tinh chỉnh và ổn định chính xác tại đáy nghiệm tối ưu.

Câu 6: Lịch tuyến tính, hàm mũ, fast và logarithmic khác nhau ra sao? Lịch nào phù hợp trong thí nghiệm?

a) Phân biệt:

Tuyến tính ($T_k = T_0 - \eta k$): Nhiệt độ hạ đều đặn; dễ rơi xuống 0 đột ngột nếu không ước lượng đúng số bước.

Hàm mũ ($T_{k+1} = \alpha T_k với 0 < \alpha < 1$): Hạ nhiệt nhanh ở giai đoạn đầu và giảm chậm dần về sau; tạo chuyển tiếp mượt mà từ khám phá sang khai thác.

Fast Annealing ($T_k = \frac{T_0}{(1+ck)}$): Giảm theo hàm hypebol; hạ nhiệt nhanh nhưng dễ rơi vào cực tiểu cục bộ nếu c lớn.

Logarithmic ($T_k = \frac{c}{ln}(1 + k)$): Được chứng minh bảo đảm hội tụ 100% về nghiệm tối ưu nhưng tốc độ giảm cực kỳ chậm, không thực tế trong ứng dụng.

b) Lịch phù hợp trong thí nghiệm: Lịch hàm mũ (Exponential) với $\alpha \in [0.99, 0.995]$ và $T_0 \in [10, 20]$ là tối ưu nhất, đạt tỉ lệ tìm thấy f=0 trên 90% trong thời gian rất ngắn.

Câu 7: Vì sao biểu diễn n-Queens mỗi cột đúng một quân giúp giảm không gian tìm kiếm?

Nếu đặt ngẫu nhiên n quân hậu lên bất kỳ ô nào trong n^2 ô: số trạng thái là $C^{n}_{n^2}$. Với n = 8: $C^{8}_{64} \approx 4,42x10^{9}$ trạng thái.

Khi dùng vector $q = [q_{0,}..., q_{n-1}]$ (mỗi cột có đúng một quân hậu tại hàng q_i): xung đột theo cột được triệt tiêu tự động 100%. Không gian tìm kiếm thu hẹp về n^n. Với n = 8, chỉ còn $8^8 = 16.777.216$ trạng thái (giảm hơn **260 lần** so với 4,42 x 10^9), giúp thuật toán hội tụ nhanh và ít nhánh vô ích.

Câu 8: Vì sao đếm xung đột theo hàng/đường chéo có thể có độ phức tạp O(n) thay vì O(n²)?

- Cách ngây thơ so sánh từng cặp đòi hỏi duyệt $C^2_n = fracn(n - 1)2$cặp, dẫn đến độ phức tạp O(n^2).

- Cách tối ưu dùng mảng tần suất:

Các ô trên cùng đường chéo chính có tổng ($i + q_i$) không đổi.

Các ô trên cùng đường chéo phụ có hiệu ($i - q_i + n$) không đổi.

Chỉ cần một vòng lặp duyệt qua n cột (O(n)) để cập nhật số lượng quân vào 3 mảng đếm.

Với mỗi nhóm có k quân, số cặp xung đột là $C^2_k = frack(k - 1)2$ (hàm comb2). Tổng xung đột được tính chỉ trong thêm O(n) bước.

Tổng độ phức tạp là O(n) + O(n) = O(n).